

Campus-Focused Digital Solutions : arX Classroom and Grants Dashboard

A PROJECT REPORT

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY :

Alakhsimar Singh (22103008)

Kanav Goyal (22103082)

Nischay Verma (22103110)

Under the supervision of

Dr. Amritpal Singh

Assistant Professor



**Department of Computer Science and Engineering
Dr. B.R. Ambedkar National Institute of Technology Jalandhar
144008, Punjab (India)
June 2026**

CANDIDATES' DECLARATION

We hereby affirm that the project titled "Campus-Focused Digital Solutions : arX Classroom and Grants Dashboard" has been developed by us and is being submitted to the Department of Computer Science and Engineering. Under the esteemed guidance of **Dr. Amritpal Singh**, we have independently created this project and it is free from any form of plagiarism.

We, the undersigned, collectively declare that this project report represents our original contributions and collective effort. Throughout its development, we have adhered to the guidelines, ethical standards, and academic integrity expected of us. We confirm that all information, data, and sources used in this project have been appropriately cited and referenced, giving proper credit to the original authors and researchers. Any external materials, including text, images, or multimedia, have been duly acknowledged and attributed.

The content of this report has not been submitted to any other university or institute for the award of any degree or for any other purpose.

Date: 3rd June 2026

Submitted by:

Alakhsimar Singh (22103008)

Kanav Goyal (22103082)

Nischay Verma (22103110)

This certifies that the statements provided by the aforementioned candidates are true and accurate to the best of our knowledge, and we recommend them for external evaluation.

Dr. Amritpal Singh, Supervisor
Assistant Professor
Department of CSE

Dr. AL Sangal
Head and Professor
Department of CSE

ACKNOWLEDGEMENT

It is often said that behind the scenes of a successful play, there are numerous individuals working tirelessly. The outcome of our project, “Campus-Focused Digital Solutions: arX Classroom and Grants Dashboard”, was only made possible through the invaluable guidance and assistance of many people. We consider ourselves incredibly fortunate to have received unwavering support throughout our project journey. We are truly grateful for the supervision and aid we have received, and we express our heartfelt gratitude to all those involved.

We deeply appreciate Professor Dr. AL Sangal, Head of the Department of Computer Science & Engineering, for his direct and indirect support.

We are grateful to the Coordinators of the Major Project (Batch-2022) for assigning us a mentor and providing various forms of assistance.

Special thanks go to our mentor, Dr. Amritpal Singh, Assistant Professor, for believing in our idea and offering valuable suggestions whenever needed. Her unwavering support in resolving our challenges has been crucial.

We are also very thankful for the continuous encouragement and guidance from all the faculty members of the Department of Computer Science & Engineering (CSE). Their valuable time and insightful suggestions have played a significant role in the successful completion of our project. Furthermore, we extend our sincere gratitude to all the laboratory staff for their timely support.

Thank you.

Alakhsimar Singh

Kanav Goyal

Nischay Verma

ABSTRACT

This report, titled “Campus-Focused Digital Solutions: arX Classroom and Grants Dashboard”, presents the design, development, and implementation of two institution-oriented web platforms developed to address specific academic and administrative requirements at Dr. B. R. Ambedkar National Institute of Technology Jalandhar. The report is organized into two major parts, with each section dedicated to a detailed discussion of one project: the Grants Dashboard and arX Classroom.

Many educational institutes rely heavily on third-party software applications to manage their administrative functions and evaluate students' assignments. While many software applications come loaded with all sorts of features, they do not usually offer the sort of flexibility that is needed by each institution. This is why leading educational institutes, like Indian Institute of Technology Bombay, Indian Institute of Technology Madras, and Indian Institute of Technology Kanpur, have developed digital ecosystems specifically for their own use.

In the first half of the report, arX Classroom is described, a lightweight and self-hosted system for teaching programming courses and performing automated code testing. The system provides instructors with the ability to generate coding tasks, perform classroom management, and evaluate students' programs by providing sandboxed execution in secure, isolated Docker environments.

In the second half of this report, we will discuss the Grants Dashboard, which is basically an application designed to enable an efficient management process for the institute's finances. Some of the key features of this application include real-time analytics, document management, financial tracking, reporting and role-based security mechanism, among others.

As mentioned, both platforms are examples of implementing contemporary methods of software engineering that include responsive design, integration with databases, authentication and authorization, scalable back-end infrastructure, and other aspects. These platforms take into consideration such factors as scalability, maintainability, usability, and economy in their development.

PLAGARISM REPORT

We have utilized Turnitin to perform a plagiarism check on our Project Report for the “A Novel PVO-Based RDH Scheme Utilizing Dual-Pixel Interleaved Data Embedding and Threshold-Driven Capacity Control” project. We express our gratitude to our mentor, Dr. Amritpal Singh, for providing guidance in this matter. The digital receipt confirming the results is presented below, indicating a plagiarism score of less than

We sincerely appreciate the support and assistance received throughout this process.

Thank you.

Alakhsimar Singh

Kanav Goyal

Nischay Verma

LIST OF FIGURES

Figure Number	Description	Page Number
Fig. 1.2.1	Prutor Coding Platform Used by IIT Kanpur, IIT Bombay, IIT Goa, IIT Roorkee, and IISc	3
Fig. 2.1.1	arX Classroom homepage	6
Fig. 2.3.1	Docker Image	9
Fig. 3.3.1	Architecture Diagram arX Classroom	12
Fig. 3.4.1	ER Diagram arX Classroom	13
Fig. 5.1.1	Student Activity Dashboard	16
Fig. 5.1.2	Sample Coding Assignment Question	17
Fig. 5.1.3	Problem Setter Control Panel	17
Fig. 8.2.1	Dashboard and Analytics Module	25
Fig. 8.2.2	Allocation Module	26
Fig. 8.2.3	Receipt Module	26
Fig. 8.2.4	Expenditure Module	27
Fig. 9.3.1	Architecture Diagram Grants Dashboard	30
Fig. 9.4.1	ER Diagram Grants Dashboard	31

TABLE OF CONTENTS

CANDIDATES' DECLARATION	(i)
ACKNOWLEDGEMENT	(ii)
ABSTRACT	(iii)
PLAGARISM REPORT	(iv)
LIST OF FIGURES	(v)
PART - I arX Classroom	
1 INTRODUCTION	2
1.1 Background of the problem	2
1.2 Comparative Study	2
1.3 Problem Statement	3
1.4 Motivation	4
1.5 Feasibility	4
2 PROPOSED SOLUTION	6
2.1 arX Classroom	6
2.2 Core Features	7
2.3 Security Architecture	8
3 TECHNOLOGY ANALYSIS	10
3.1 Methodology	10
3.2 Technology Stack	10
3.3 Architecture Diagram	11
3.4 Database Schema	13
4 ECONOMIC ANALYSIS	15
5 RESULTS AND DISCUSSION	16
5.1 Results	16

5.2 Challenges and Solutions	18
6 CONCLUSION	19
PART - II Grants Dashboard	
7 INTRODUCTION	21
7.1 Background of the problem	21
7.2 Comparative Study	21
7.3 Problem Statement	22
7.4 Motivation	22
7.5 Feasibility	23
8 PROPOSED SOLUTION	24
8.1 Grants Dashboard	24
8.2 Core Features	24
9 TECHNOLOGY ANALYSIS	28
9.1 Methodology	28
9.2 Technology Stack	28
9.3 Architecture Diagram	29
9.4 Database Schema	30
10 ECONOMIC ANALYSIS	32
11 RESULTS AND DISCUSSION	33
11.1 Results	33
11.2 Challenges and Solutions	33
12 CONCLUSION	34
REFERENCES	35

PART – I
arX Classroom

1. INTRODUCTION

1.1 Background of the problem

Programming-based teaching has become a very crucial aspect of modern-day computer science education programs [1]. Educational institutions now use digital platforms for distributing assignments, coding exercises, lab work, and evaluation purposes [2]. Nonetheless, many educational institutions still use external platforms for managing their programming-based educational activities such as Google Classroom, LeetCode, HackerRank, and Codeforces [3].

Such digital platforms do offer general functionalities but lack institution-specific customizations, flexible deployments, centralized administration, and workflow integrations. Another important challenge faced by instructors in teaching programming courses lies in the manual evaluation of students' assignments, which is tedious and hard to manage [4]. This is particularly true when dealing with courses having many students.

The increasing significance of programming education as well as competitive coding has made it necessary to develop secure and automated ways of assessing students' programs to make the process easier and more effective. Institutions have also become demanding of customized platforms with the ability to perform institutional workflow integrations among other things.

This necessitates the creation of institution-based platforms that can assist in automated assessment and evaluation of programming assignments, and secure program executions amongst other functionalities required in educational institutions such as Dr. B. R. Ambedkar National Institute of Technology Jalandhar.

1.2 Comparative Study

There are a variety of existing websites and online systems for code training and competitive coding. Websites such as LeetCode, HackerRank, or Codeforces offer testing of source codes, multilingual support, and even environments for competing in programming. Such services provide a standard for automated judging of code and its assessment.

But in many cases, commercially available and publically accessible websites do not allow much personalization of institution-related academic workflow as well as control of the deployment

environment. In this regard, institutions may want to develop self-hosted versions that would be able to perform customized assignment creation, as well as handle secure code assessment in a personalized manner.

Systems such as DOMjudge or Sphere Online Judge have demonstrated success [5] in utilizing systems for judging code and performing automated assessment in academic and competitive contexts. The key aspect here is security, scalability of testing cases, and verdicts generation.

Several IITs have already developed and deployed their own in-house coding platforms for programming labs, assignments, and assessments. This validates the feasibility of institution-developed solutions and highlights the benefits of customization, ownership, and reduced operational costs compared to commercial alternatives.

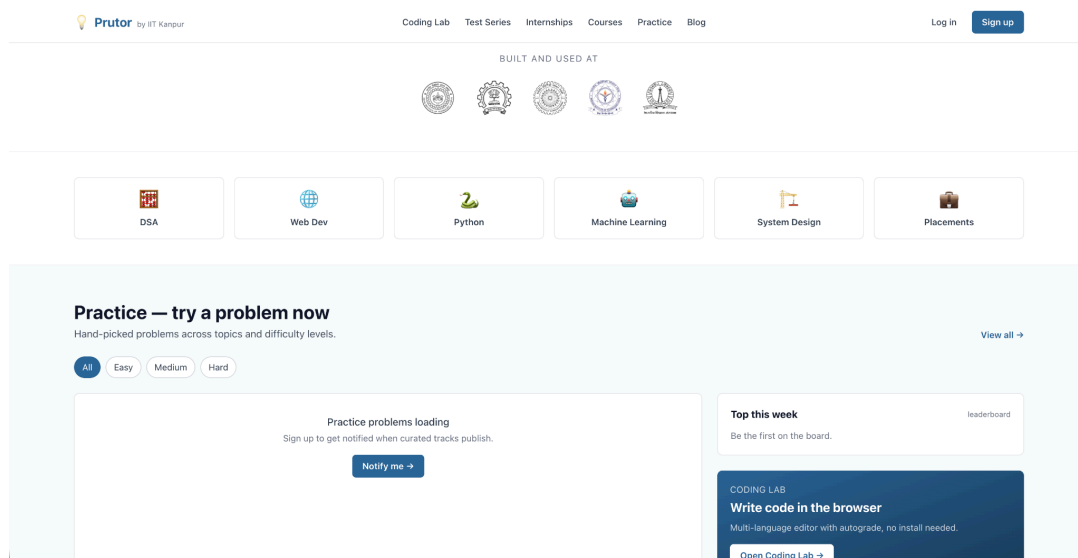


Fig. 1.2.1 Prutor Coding Platform Used by IIT Kanpur, IIT Bombay, IIT Goa, IIT Roorkee, and IIT Sc

Based on the study of existing systems, the demand for lightweight, self-hosted, and institution-centered services is rising [6] due to their usability, security, and scalability. arX Classroom was designed with these aspects in mind.

1.3 Problem Statement

Programming courses require an efficient system for creating coding assignments, conducting assessments, and evaluating student submissions at scale. Existing approaches often involve manual evaluation, fragmented tools, or costly third-party platforms, leading to increased

administrative effort and inconsistent feedback. Therefore, there is a need to build a centralized, self-hosted platform that enables educators to create programming problems, manage assignments, automate code execution and evaluation, and monitor student performance through an integrated interface.

1.4 Motivation

The need for a solution to enhance the process of learning through coding was what made the creation of arX Classroom necessary, together with the problems experienced during manual code grading within institutions.

Dependence on external platforms hinders the capacity to customize, deploy, and manage academic processes. Professors are required to use a lot of time manually compiling and grading programming exercises, which is very time-consuming in cases where there are many students per course.

The development of an in-house automation system that allows more consistent grading, less workload for professors, and more immediate feedback from students was one of the reasons behind creating arX Classroom. Another motivation included gaining experience with software development techniques that include full-stack web development, safe containerization, role-based authentication, and academic workflow management.

In summary, arX Classroom intends to develop a flexible, cost-efficient, and easily deployable alternative to other similar commercial tools used in programming evaluations.

1.5 Feasibility

1.5.1 Technical Feasibility

The arX Classroom application is designed using common open-source software tools such as Next.js, React, TypeScript, SQLite, and Docker [7]. Such tools help provide scalability, maintainability, platform independence, and community backing.

The isolation of containerized code execution involves running the program inside a separate Docker container [8], thereby creating runtime environments that are isolated from system

access, have limited resource utilization, and execution time requirements. This helps ensure the feasibility of the technical solution.

1.5.2 Economic Feasibility

Platform economics are sustainable as this solution makes use of only free and open source solutions, which exclude any need to pay for licenses and subscriptions, which are common charges for many commercial coding test platforms.

Deployment demands are not high and can be achieved by leveraging relatively cheap virtual private servers or institutionally-owned servers that include Docker capabilities.

1.5.3 Operational Feasibility

The platform offers a responsive and intuitive interface that can be utilized by students, instructors, and administrators with limited need for technical skills.

Evaluation processes are automated to save time on grading and provide greater consistency in assessing programming assignments. Academic workflows are streamlined through role-based access controls, thus ensuring efficient management of the platform resources.

This approach helps not only maintain the platform more effectively but also allows for further improvement of its functionalities.

2. PROPOSED SOLUTION

2.1 arX Classroom

arX Classroom is a web-based, self-hosted programming education and automated code assessment platform developed specifically for educational institutions. It is designed to help instructors create and manage coding problems, assignments, and programming exercises while enabling students to submit solutions in multiple programming languages such as C, C++, and Python. The platform automatically evaluates submitted programs by compiling and executing them inside secure Docker containers, running them against predefined test cases, enforcing execution constraints such as time and memory limits, and generating verdicts and scores. Built using modern technologies including Next.js, React, TypeScript, SQLite, and Docker, arX Classroom incorporates role-based access control for students, problem setters, and administrators, along with dashboards for monitoring submissions and performance analytics. The system aims to provide institutions with a customizable, cost-effective alternative to external platforms like HackerRank and LeetCode by offering secure code execution, automated grading, centralized administration, and full control over academic workflows.

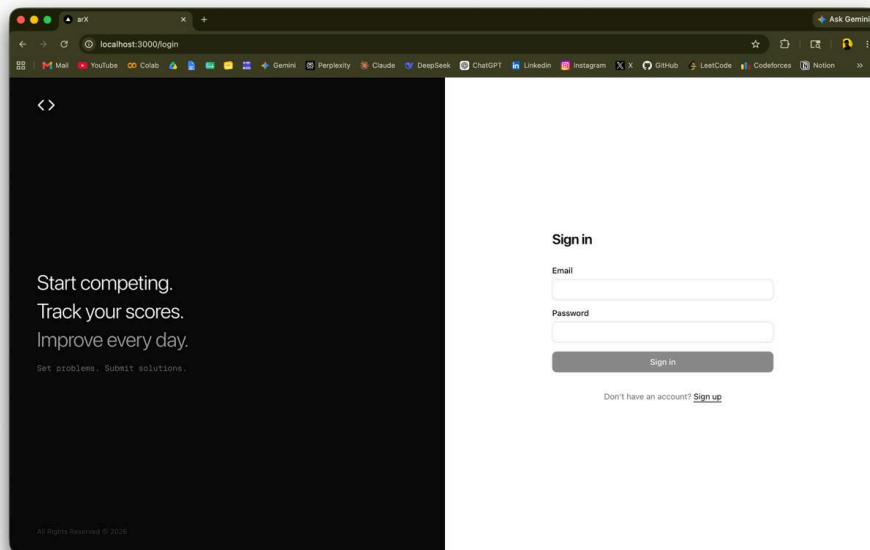


Fig. 2.1.1 arX Classroom homepage

2.2 Core Features

2.2.1 Authentication System

The platform ensures that authentication and authorization operations are secure by leveraging the email-password mechanism along with session-based authentication [9]. Session management is performed via signed cryptic tokens while role-based authorization facilitates separation of duties [10] between students, setters, and admins.

Roles supported on the platform are:

1. Solver – Solve programming problems
2. Setter – Develop and manage programming problems
3. Admin – Configure permissions and workflow

2.2.2 Problem Management System

The platform includes an easy-to-use problem management system where educators can manage problems, their descriptions, and other attributes such as:

1. Time constraints
2. Memory constraints
3. Public and hidden testcases
4. Weighted grading schemes
5. Visibility options

This allows assignments, lab classes, and programming sessions.

2.2.3 Multi-Language Code Execution

arX Classroom supports several programming languages such as C, C++ and Python utilizing dedicated containers [11] in Docker technology stack. Specifically, each programming language is tied to an individual execution environment performing the task of compiling and running the code.

2.2.4 Automated Judging Engine

The judging engine on the platform handles the submission and performs evaluation [12] by going through the process of compilation, testcase execution, and verdict assignment.

The evaluation engine executes tasks including:

1. Log file generation
2. Execution time and memory usage checks per testcase
3. Generation of verdict based on execution and score

2.2.5 Activity Dashboard

The system also has a dashboard which will help educators and administrators gain an understanding of:

1. Submissions
2. Students' participation
3. Performance per problem
4. Evaluation results

These analytics will aid instructors in assessing students' performance and problem difficulty levels.

2.2.6 Administrative Controls

The admin portal is used to administer access permissions across the system. Through this, administrators are able to manage users who are authorized setters.

2.3 Security Architecture

2.3.1 Container-Based Isolation

Submitted code will run in a completely isolated environment, the Docker container, [13] which comes with the following features:

1. Lack of external network connectivity
2. Restrictive processes execution capability

3. Limited resource consumption capabilities such as CPU and memory usage restrictions
4. Limitations on execution time

This isolation approach will help us avoid having any kind of malicious software interacting with our hosting systems.

☐	Name	Tag	Image ID	Created	Size	Actions
☐	☐ judge-c	latest	00275363c478	2 minutes ago	1.9 GB	  
☐	☐ judge-cpp	latest	2ee22017bac9	2 minutes ago	1.9 GB	  
☐	☐ judge-python	latest	b5f26c0b53bb	2 minutes ago	211.76 MB	  

Fig. 2.3.1 Docker Image

Each programming language on the platform has a different Docker image, as shown in Fig. 2.3.1. Since the C and C++ execution environment contain the complete compiler chain as well as libraries and dependencies, their respective images are of size 1.9 GB each. However, the Python execution environment only needs an image of size 211.76 MB. This is because of the execution mode that Python follows, which involves an interpreter.

2.3.2 Password Security

Passwords will be stored in a hashed form [14] after running them through secure hashing algorithms using a random salt for improved security.

2.3.3 Session Management

In order to prevent possible client-side attacks like XSS, we will be implementing HTTP only session cookies to manage authentication status [15].

2.3.4 Route Protection and Middleware

We use middleware to verify permissions before allowing access to any sensitive endpoints.

3. TECHNOLOGY ANALYSIS

3.1 Methodology

The design of arX Classroom involves adopting a modular full-stack application that is centered around scalability, maintainability, secure execution of code, and effective academic workflow management. The platform is aimed at providing a unified environment for educational purposes allowing teachers to create tasks, conduct evaluations, and track student progress in one place.

arX Classroom features a component-oriented architecture where various components including the frontend, backend services, database, and code execution containers operate as separate but interconnected modules of the system. This approach helps to increase maintainability of code, improve system performance, and facilitates further updates and scaling of system components depending on user traffic.

To facilitate a secure and fast evaluation process, the platform has a submission queuing system that deals with all submitted code. Programs are added to a queue for execution in Docker containers in order to prevent system congestion and to ensure stable operation. Thus, even when many users submit tasks at the same time, the system provides timely execution of the program, avoids potential crashes, and ensures correct evaluation.

3.2 Technology Stack

3.2.1 Next.js and React

Next.js and React were selected due to their component-based architecture [16], scalability, and strong ecosystem support for modern web application development.

Next.js additionally provides integrated routing, API support, server-side rendering, and optimized application [17] performance within a unified framework.

3.2.2 Typescript

TypeScript improves maintainability and code reliability through static typing [18], compile-time validation, and enhanced development tooling support.

3.2.3 SQLite

SQLite was selected to prioritize lightweight deployment [19], simplicity, and reduced configuration overhead while providing sufficient performance for institutional academic workloads.

3.2.4 Docker-Based Execution Environment

Docker provides isolated execution environments [20] for submitted programs, ensuring secure automated evaluation through process isolation and controlled resource utilization.

3.2.5 Markdown-Based Content Rendering

The platform uses react-markdown for rendering structured problem statements with support for headings, code blocks, lists, and formatted instructional content.

3.3 Architecture diagram

The architecture of the ARX Classroom revolves around the role-based coding assessment platform, allowing secure code execution, problem management, and performance assessment. The first phase includes user authentication; users log into the platform using an authenticated service. Depending upon the role defined in the process of logging into the platform, users will be classified either as Administrators or Solvers. The use of role-based access control allows us to assign permissions based on the task the individual user will have to perform.

The administrators play the role of configuring and managing the application. They will have the ability to manage roles and permissions, update the setter allowlist, create coding problems, set problem statements, provide test cases, and set the execution constraints. All problem-related data are stored in a SQLite database and thus act as the main storage area for the problem and other data in the platform. The administrators thus make the problem setting and management process more efficient.

Solvers get a fully automated code evaluation pipeline from the architecture. Once they choose a task and submit their solution, it goes via a Next.js API route which prepares an isolated Docker execution container for the code. Then, the source code of the solution is built in such a safe environment to prevent any impact on the hosting computer. In case the build fails, all the

compilation errors will be shown to the user. In the case when it was successful, the code gets run against test cases using the timing and memory usage tracking. Finally, a verdict and scoring of the solution is generated and saved together with submission details and delivered to the user. Moreover, statistics and analytics related to submissions and tasks can be provided to make data-driven evaluations possible.

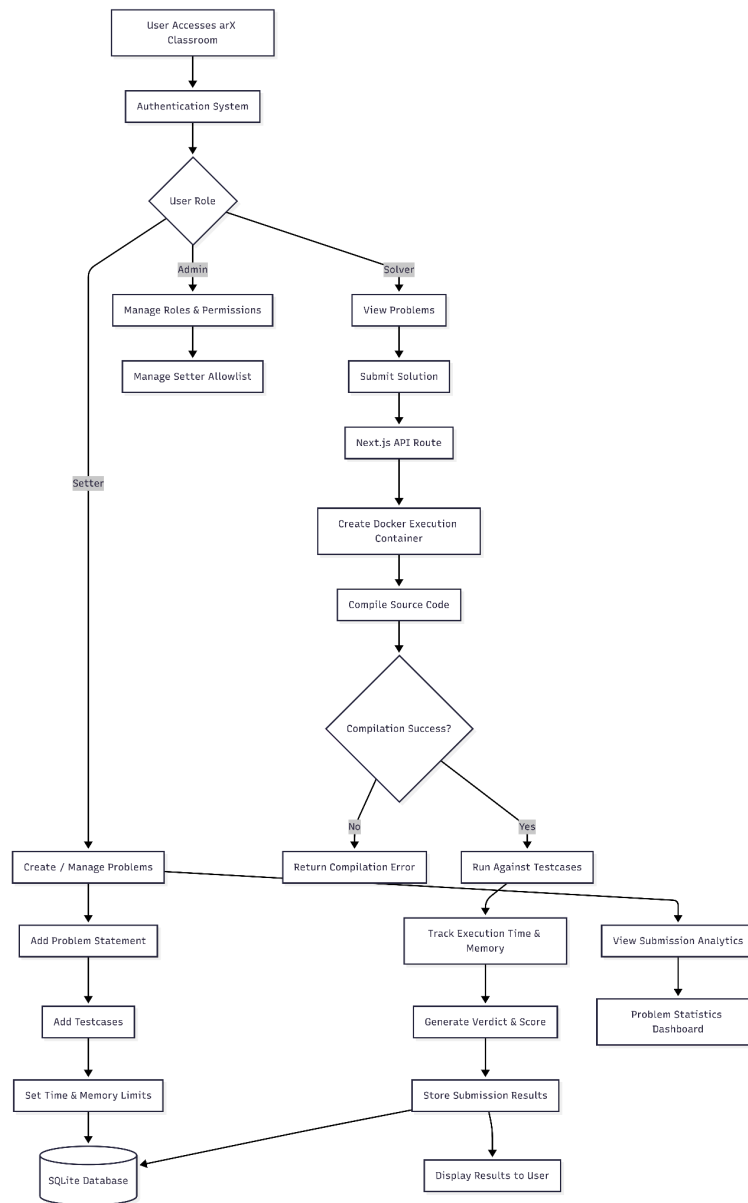


Fig. 3.3.1 Architecture Diagram arX Classroom

3.4 Database Schema

The ER diagram below displays the structure of the database that was developed in accordance with the programming assessment platform architecture and relations between basic entities. The entity USER plays an important part in the structure as it stores all the data related to authentication of registered users and their roles within the system. All authorized users have access to the possibility to establish SETTER profiles, who are responsible for the development of programming problems. Each setter is able to create as many PROBLEMS as he/she wishes to. Every problem consists of specific details such as the statement, constraints, limits of program execution, and its visibility.

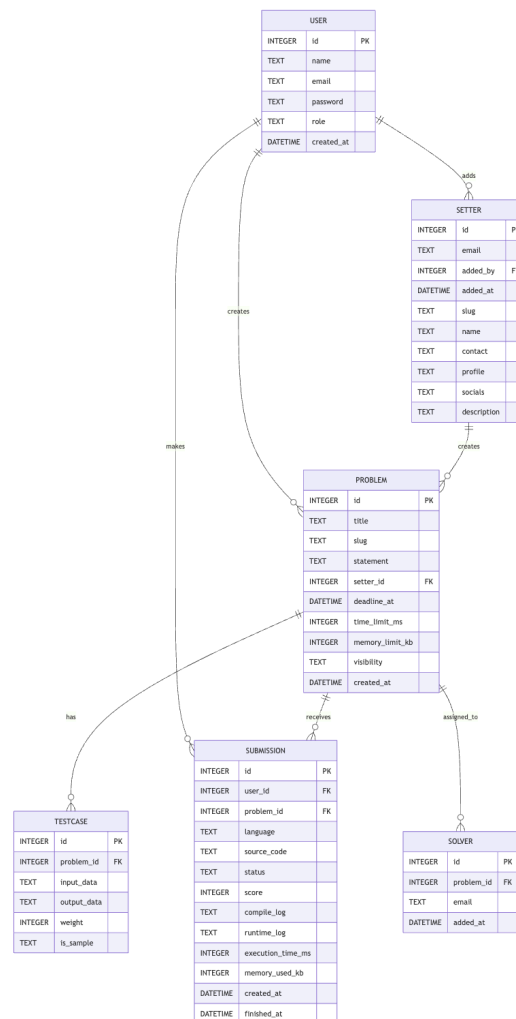


Fig. 3.4.1 ER Diagram arX Classroom

The interaction of students with the system is conducted via SUBMISSIONs, that consist of additional data about a user, his/her selected programming language, code, execution log, received score, memory consumed, and execution time. A single problem may be submitted multiple times by several users at once. The SOLVER entity is needed for creating a relation between problems and their solvers. Thus, the database structure enables the process of effective users management, problems development, automatic code evaluation, and submission tracking.

4. ECONOMIC ANALYSIS

arX Classroom uses entirely free software and technologies, making it a zero cost proposition for organizations compared to proprietary coding evaluation systems.

It needs a Node.js based server setup with Docker support for containerization. Deployment could be done on low-cost virtual private servers or organizational setups.

In light of the fact that the platform should be deployed within existing organizational frameworks, there is no deployment cost at all.

When compared to commercial systems like HackerRank and Codility, the platform offers better flexibility in deployments, institutional control of data, and customization options without any subscription cost.

This makes the platform more efficient in terms of reducing efforts from instructors due to automated features of compilation, testing, scoring, and instantaneous feedback options.

5. RESULTS AND DISCUSSION

5.1 Results

The deployment of arX Classroom served to successfully show that automatic and secure grading of programming tasks can be achieved through isolation in containerized execution environments.

Outcomes observed include:

1. Automatic and successful evaluation of programming tasks in C, C++, and Python
2. Testcase-based assessment with public and hidden testcases
3. Execution-time and memory constraint enforcing
4. Containerized execution based on Docker
5. Role management for administrators and setter authorization
6. Activity dashboards for monitoring participant and problem performance
7. Scalable system architecture capable of supporting up to 500 concurrent users

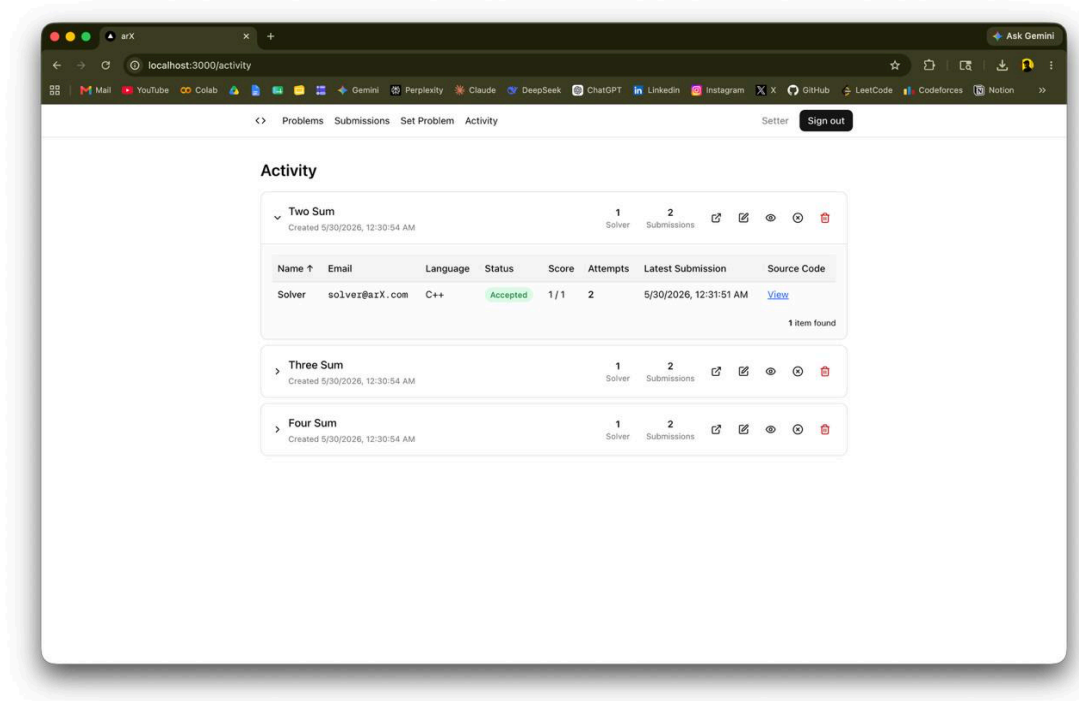


Fig. 5.1.1 Student Activity Dashboard

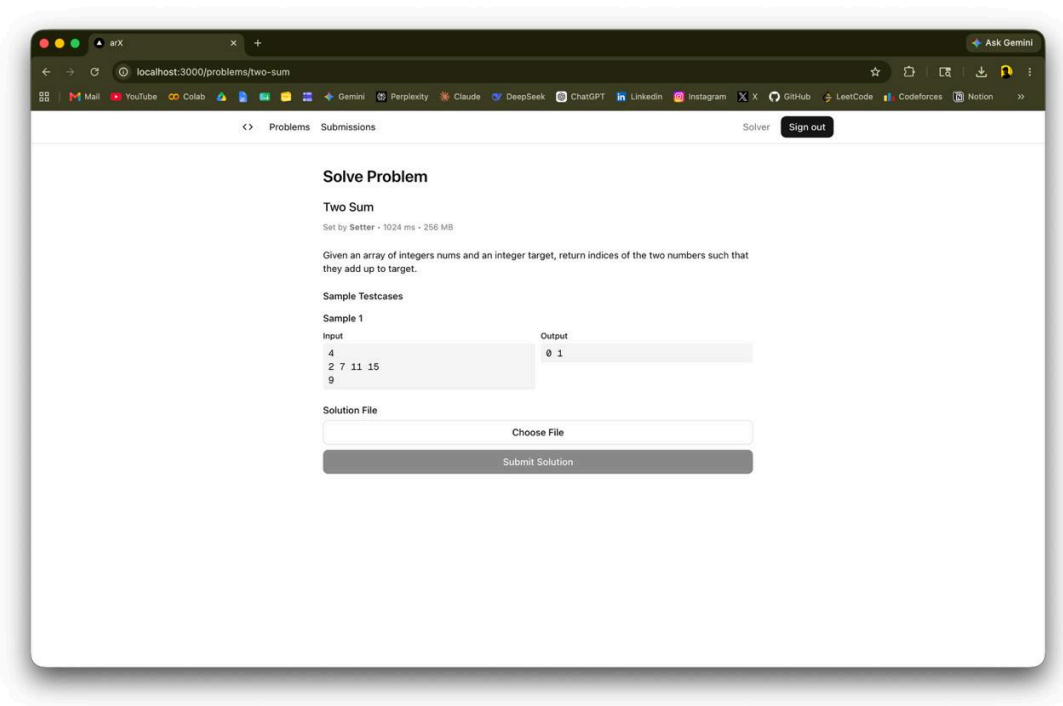


Fig. 5.1.2 Sample Coding Assignment Question

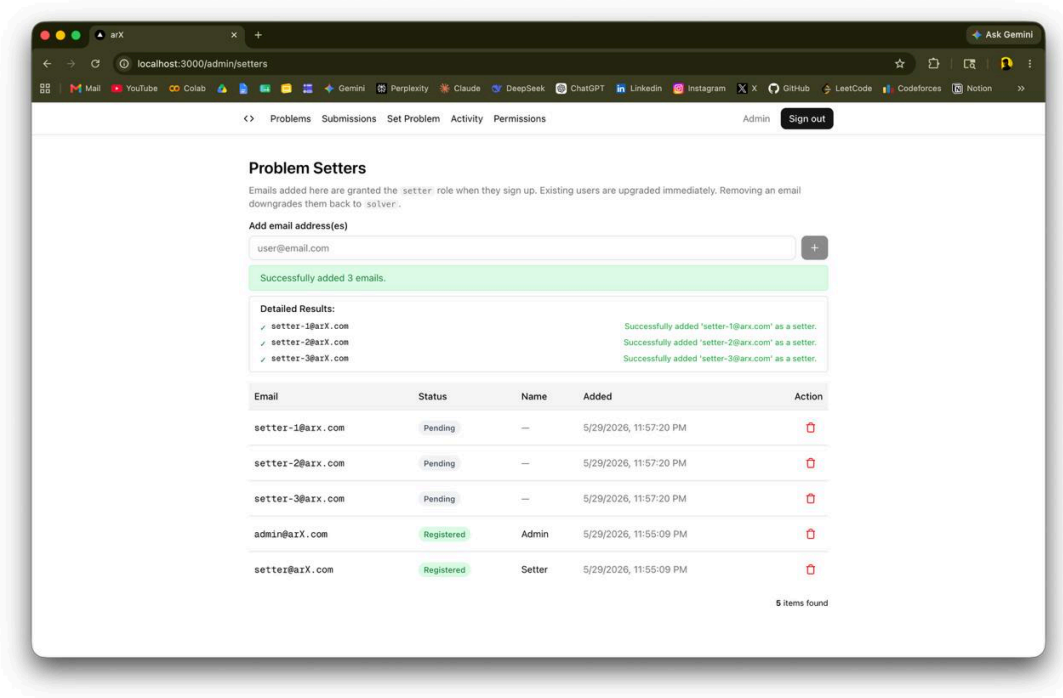


Fig. 5.1.3 Problem Setter Control Panel

5.2 Challenges and Solutions

5.2.1 Cross-Origin Communication Issues

During the initial stages of development, communication between the frontend and backend services encountered restrictions imposed by browser cross-origin policies. These restrictions affected secure data exchange and API communication between different application layers.

To address this issue, comprehensive Cross-Origin Resource Sharing (CORS) middleware was implemented with support for credential handling, controlled origin validation, and secure request management. This ensured stable and secure communication between the frontend interface and backend services.

5.2.2 Resource Consumption of Docker Containers

During testing, improper handling of Docker execution containers occasionally resulted in temporary resource accumulation and increased system overhead. Continuous creation of containers without proper cleanup affected overall resource efficiency.

To resolve this issue, automated container lifecycle management mechanisms were implemented to ensure that execution containers are automatically terminated and removed immediately after program execution. This improved resource utilization and maintained stable platform performance during repeated submission evaluations.

5.2.3 Secure Testcase Storage

Initially, testcase data was stored using plain files, which introduced security concerns related to unauthorized access and potential exposure of hidden evaluation testcases.

To improve security, testcase information was migrated to structured database storage with restricted application-level access controls. This approach ensured secure testcase management while preventing direct unauthorized access to hidden evaluation data.

6. CONCLUSION

arX Classroom successfully demonstrated the feasibility of a lightweight and self-hosted platform for programming education and automated code assessment.

The platform implemented secure containerized execution, automated testcase-based evaluation, role-based workflows, and multi-language support suitable for institutional academic environments.

The project showcased modern full-stack software engineering practices including component-based frontend development, secure authentication mechanisms, scalable backend architecture, responsive interface design, and containerized execution environments.

The adoption of open-source technologies ensures maintainability, deployment flexibility, reduced operational cost, and scalability for institutional usage.

The implementation demonstrated improvements in workflow automation, evaluation consistency, operational efficiency, and academic scalability compared to traditional manual assessment approaches and dependency on external generalized platforms.

Possible future enhancements for arX Classroom include:

- Support for additional programming languages such as Rust, Go, and JavaScript
- Integration of plagiarism detection systems
- Interactive debugging environments
- Competitive contest mode with rankings and leaderboards
- Peer review and instructor feedback workflows
- Integration with institutional learning management systems and academic portals

PART – II

GRANTS DASHBOARD

7. INTRODUCTION

7.1 Background of the problem

Educational institutions today widely employ the use of advanced technologies [21] to enhance the efficiency and transparency of their administrative tasks. Nevertheless, many educational establishments today employ outdated workflows and spreadsheets for managing their financial activities.

For instance, at Dr. B. R. Ambedkar National Institute of Technology Jalandhar, most financial activities can be handled by employing disconnected reporting systems and traditional administration methods. These methods are cumbersome, ineffective and inefficient in scaling up the operations of an organization and prone to errors.

Financial transparency and accountability in grant-funded organizations is extremely important, especially since the institutions have to keep proper track of their allocations, expenditure, and receipts and also have documents proving that. Financial activities require more than traditional methods to manage effectively.

Thus, the problems associated with financial administration in educational institutions today pose a great need for a reliable, efficient and easily manageable platform for handling financial activities.

7.2 Comparative Study

There are quite a few financial management solutions available today. Tools like QuickBooks and Tally Solutions offer solutions [22] related to transactions management, accounting, reporting, and financial analytics. The software solutions mentioned above are very popular in commercial settings owing to their reliability and feature sets.

Nevertheless, most commercial solutions do not cater specifically to the needs of institutions in terms of allocating resources, accounting of department spending, maintaining sanction data, performing financial analyses, and providing reports according to the institutions' needs. Besides that, most solutions require licensing fees on a regular basis and allow for a limited degree of customization.

Most modern solutions for financial management incorporate dashboards, responsive design, and real-time analytical tools [23]. Popular programming languages include React and TypeScript, along with other interactive tools used in visualizations.

Therefore, it could be stated that there is a need for highly customized, scalable, and institution-oriented solutions that would integrate usability, scalability, reporting features, and administration into one solution.

7.3 Problem Statement

To design and develop a secure web-based Grants Dashboard that enables institutions to efficiently manage and monitor financial allocations, receipts, expenditures, and supporting documentation through centralized record management, analytical dashboards, reporting tools, and role-based access control.

The platform aims to improve financial transparency, reduce manual administrative workload, and support streamlined grant-related workflows within institutional environments.

7.4 Motivation

The need for the implementation of the Grants Dashboard stems from the necessity to increase financial transparency, accountability, and efficiency within the process of grant-related transactions at institutions.

Old-fashioned solutions based on the use of spreadsheets and manual accounting practices are not only ineffective but are also difficult to implement on large scales. It is necessary to have centralized digital tools able to manage financial information efficiently.

Moreover, the creation of a custom-made software solution to be applied specifically in institutions was another motivating factor. The project also allowed me to work on modern full-stack solutions related to frontend responsive design, RESTful architecture, data storage, visualization of results and access control.

The software solution is intended to provide an affordable and scalable approach.

7.5 Feasibility

7.5.1 Technical Feasibility

The Grants Dashboard leverages commonly used open-source frameworks such as React, TypeScript, PHP, and MySQL [24] that are scalable, reliable, have active communities, and ensure fast performance of web-based administrative applications.

The frontend approach allows the creation of adaptive and interactive UIs [25] whereas the backend API ensures proper data handling, business rules execution, and database interaction. The choice of frameworks ensures modularity, maintainability, and scalability of the solution.

7.5.2 Economic Feasibility

The development project is economically viable because it involves the use of free and open-source technologies. Thus, no licensing fees related to commercial financial management applications will arise.

The deployment requires relatively few resources. They can easily be provided with the help of standard PHP-based hosting services or even low-cost virtual private servers. Thus, implementation costs will be greatly reduced, and institutions will retain full control over the application.

7.5.3 Operational Feasibility

User-friendliness and operational simplicity are among the key design principles in developing the platform. The ability to work with the responsive interface will allow institutional administrators to handle financial workflows without the need for professional skills and extensive training.

The platform's capabilities of centralized financial tracking and reporting with proper categorization help in improving the efficiency of the administration.

8. PROPOSED SOLUTION

8.1 Grants Dashboard

Grants Dashboard is an online centralised financial management system that helps automate the processes associated with allocating, collecting, spending money and reporting on grants at any institution. This system facilitates maintaining financial records, monitoring budget usage, as well as analysing finances with the help of various visualised reports and dashboards.

The online system is built in such a way that allows the administration to organise grant financial flows by creating online financial flows. Information on financial activity is presented via several modules : allocation, collection, expenditure, analytics, and reports.

The dashboard of the online system gives real time financial insights [26] with various types of data analytics helping institutions track budget usage and expenditure distributions among departments and categories. Charts and graphs, categorised report generation, spreadsheet export feature help with financial analytics and report preparation.

Documents concerning financial flows, such as sanction orders, bills, vouchers and their attachments are stored in a safe place with permission-based access control systems. It means that authorised persons can access the information and make changes if necessary.

Thus, Grants Dashboard is a solution-oriented approach to financial management in any institution.

8.2 Core Features

8.2.1 Dashboard and Financial Analytics Module

The dashboard acts as a central hub displaying financial activities of the institution through interactive analytics visualization and real-time summaries.

Components of the module include:

1. Analysis of receipts versus expenditures
2. Determination of financial distributions by category
3. Summaries of balances

4. Financial scaling format
5. Interactive visualization and graphs

Visualization features are provided via Recharts [27] to enhance readability and financial insights.

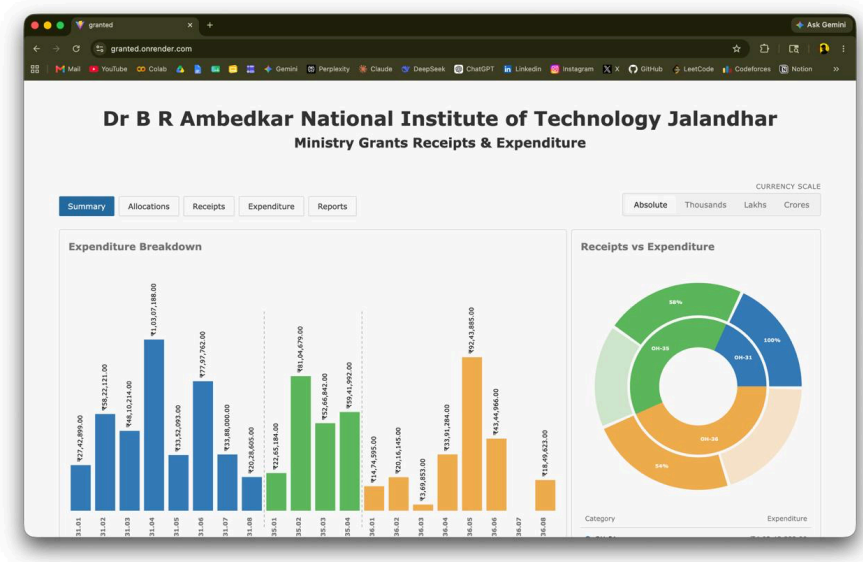


Fig. 8.2.1 Dashboard and Analytics Module

8.2.2 Allocation Management Module

This module allows administrators to allocate and manage budgets across various financial categories.

The following features can be found within the module:

1. Allocation reference numbers
2. Categorized funds allocation
3. Allocation by date
4. Institutional budget allocation workflow

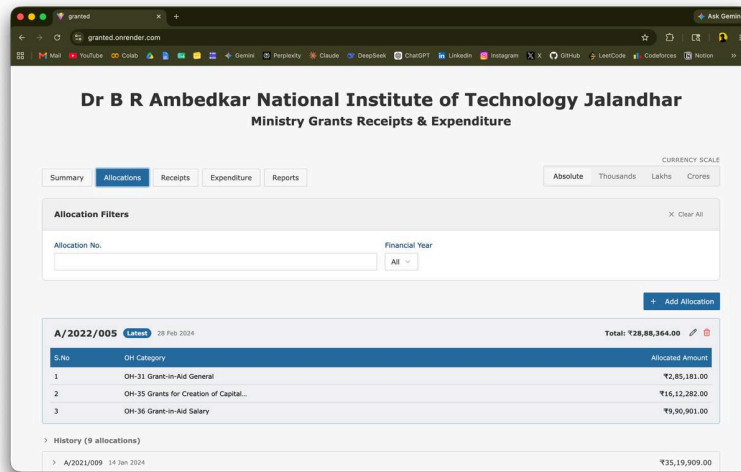


Fig. 8.2.2 Allocation Module

8.2.3 Receipt Management Module

This module handles the management of financial records received from institutional grants and funding.

Included features are:

1. Sanction order references
2. Category assignments
3. Dates of receipt
4. Attachments that are documents in PDF format

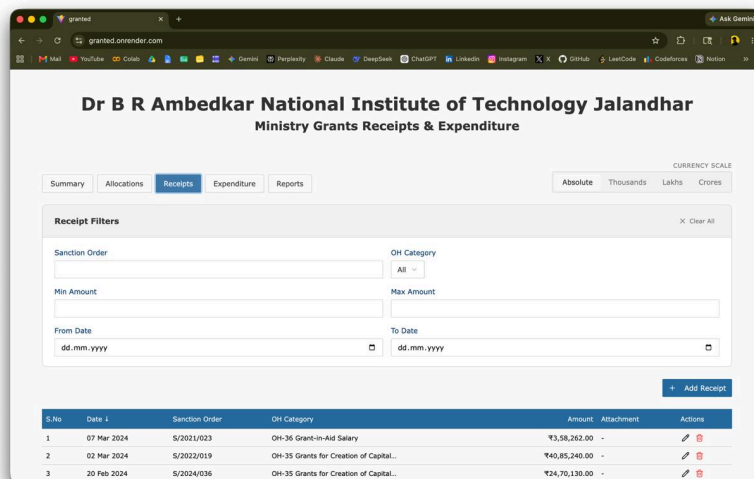


Fig. 8.2.3 Receipt Module

8.2.4 Expenditure Tracking Module

This module facilitates efficient tracking of expenditures incurred by the institution.

Included features in this module are:

1. References to bills and vouchers
2. Expenditures categorized by department
3. Classification of sub-categories
4. Description of expenditures

The screenshot displays the 'Expenditure Tracking Module' interface for the 'Dr B R Ambedkar National Institute of Technology Jalandhar'. The page title is 'Ministry Grants Receipts & Expenditure'. The interface includes a navigation bar with tabs for 'Summary', 'Allocations', 'Receipts', 'Expenditure' (selected), and 'Reports'. A 'CURRENCY SCALE' dropdown is set to 'Absolute'. Below the navigation bar is the 'Expenditure Filters' section, which contains fields for 'Bill No.', 'Voucher No.', 'OH Category' (dropdown), 'OH Sub-Category' (dropdown), 'Min Amount', 'Max Amount', 'From Date' (dd.mm.yyyy), and 'To Date' (dd.mm.yyyy). A '+ Add Expenditure' button is located at the bottom right of the filters. Below the filters is a table with the following columns: 'S.No', 'Date', 'Bill No.', 'Voucher No.', 'OH Category', 'OH Sub-Category', 'Department', 'Amount', 'Attachment', and 'Actions'. The table contains one row of data: '1', '11 Mar 2024', '8/2022/034', 'V/2024/034', 'OH-31 Grant-in-Aid General', '31.06 Other Expenses', 'Physics', '₹1,63,729.00', and an empty attachment field.

S.No	Date	Bill No.	Voucher No.	OH Category	OH Sub-Category	Department	Amount	Attachment	Actions
1	11 Mar 2024	8/2022/034	V/2024/034	OH-31 Grant-in-Aid General	31.06 Other Expenses	Physics	₹1,63,729.00		

Fig. 8.2.4 Expenditure Module

8.2.5 Reporting and Export Module

This module facilitates the generation of financial reports using different filters, such as category, department, date, and expenditure types.

This platform uses SheetJS for generating Excel spreadsheets [28] for downloading purposes.

9. TECHNOLOGY ANALYSIS

9.1 Methodology

Grants Dashboard development was conducted following a modular approach for full-stack web development aimed at maximizing its scalability, maintainability, and ability to manage institutional financial flows efficiently.

The system uses a three-layered architecture [29] composed of the frontend presentation layer, the backend application layer, and the relational database layer. In particular, the former deals with user interaction and analytical visualization, whereas the latter includes request processing, validation, business logic operation execution, and secure data exchange with the database layer.

During the development stage, requirement analysis, database schema design, API development, frontend creation as a responsive web interface, integration of reporting and analytics modules, testing, and deployment optimization were executed. Different workflows related to financial processes like allocations, receipts, expenditures, and reporting were encapsulated into different modules for better scalability and maintainability.

The backend was developed using a RESTful API architecture [30] allowing CRUD operations with all financial entities. In addition, various mechanisms for secure handling of files and access control were also designed to facilitate management of institutional financial information and associated files.

Finally, some analytical tools like dashboards and reporting systems along with spreadsheet export capabilities were also included into the system.

9.2 Technology Stack

9.2.1 Frontend Technologies

The frontend development in the Grants Dashboard uses React 19, TypeScript, and Vite. React allows the development of components for user interfaces, whereas TypeScript adds better maintainability through static typing. Vite ensures that the platform has optimized performance and efficiency.

9.2.2 Styling and User Interface

The application makes use of Tailwind CSS 4, shadcn/ui, and Radix UI [31] for effective development of a responsive interface. The technologies allow for reusable UI components to be created fast during development and implemented effectively throughout the interface.

9.2.3 Backend Technologies

The backend system of the platform uses PHP 8.x and is structured around a RESTful API design with PDO [32]. The backend has capabilities to securely communicate with databases, manage requests effectively, validate information, and perform other necessary tasks.

9.2.4 Database System

MySQL 8.x is used as the platform's main relational database [33] due to its reliability, ability to handle concurrent transactions, relational integrity, and effective query processing necessary for financial institutions.

9.2.5 Visualization and Reporting

Recharts is used for visualizing data on analytical dashboards [34], and SheetJS is utilized for generating spreadsheets compatible with Excel.

9.3 Architecture Diagram

The Grant Dashboard architecture has been created with the purpose of providing a platform where one can manage their financial activities related to grant allocation, fund receipt, expenditure, and reporting. The first process in the architecture is the authentication of the user and verification of the permissions to access financial information and conduct operations. Once the user has passed through the authentication process, he/she will have access to the unified dashboard from which one accesses modules related to grants.

The platform consists of four functional modules. These modules include Allocation, Receipt, Expenditure, and Reports & Analytics. While the allocation module helps administrators to create and edit their allocations, receipt helps one to keep the records and upload relevant files. Likewise, the expenditure module is concerned with recording expenditure together with relevant

invoices and vouchers. In the platform, all the financial and related information is kept using the MySQL database where one can maintain data consistency and reliability. Meanwhile, communication between the user interface and the database is facilitated using PHP REST API.

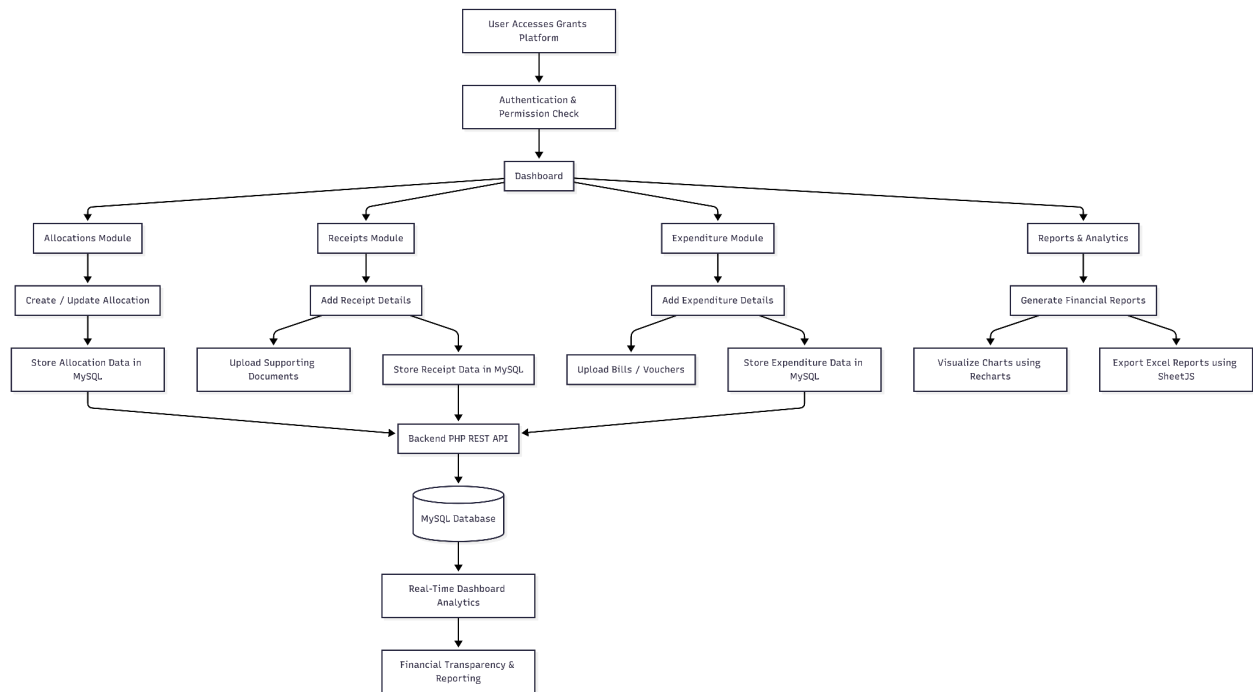


Fig. 9.3.1 Architecture Diagram Grants Dashboard

In order to aid decision-making and financial oversight, the Reports & Analytics module provides complete financial reports and visualizations. Graphs will be generated through the use of Recharts, which allows for the analysis of funding statistics through an interactive dashboard. In addition, the exportation of reports in the Excel format using SheetJS is possible, which allows for easy distribution of information. With the help of real-time analytics, data storage, and report generation, the Grants Dashboard facilitates more efficient grant administration processes.

9.4 Database Schema

The ER diagram below shows the structure of the database of the Grants Dashboard application. There are four main entities in the database. These include Receipts, Expenditures, Allocations, and Allocation_Amounts. The Receipts entity contains information about the receipts of the

funds in the form of grant sanction orders, types, amounts, documents, and dates. Expenditures entity consists of expenditures made and their details such as bill number, voucher number, department making payment, expenditure type, and related attachment.

Allocations is the entity which stores sanctioned allocations and acts as a parent table for the category-wise allocations. Allocation_Amounts entity holds the amounts allocated under various grant categories. It is connected to the Allocations entity via a foreign key relation. It creates a one-to-many relationship as one allocation can have several category-wise allocation records. Foreign keys are used for referential integrity with a cascaded delete feature. That means that deleting any allocation will result in the deletion of associated allocation amounts.

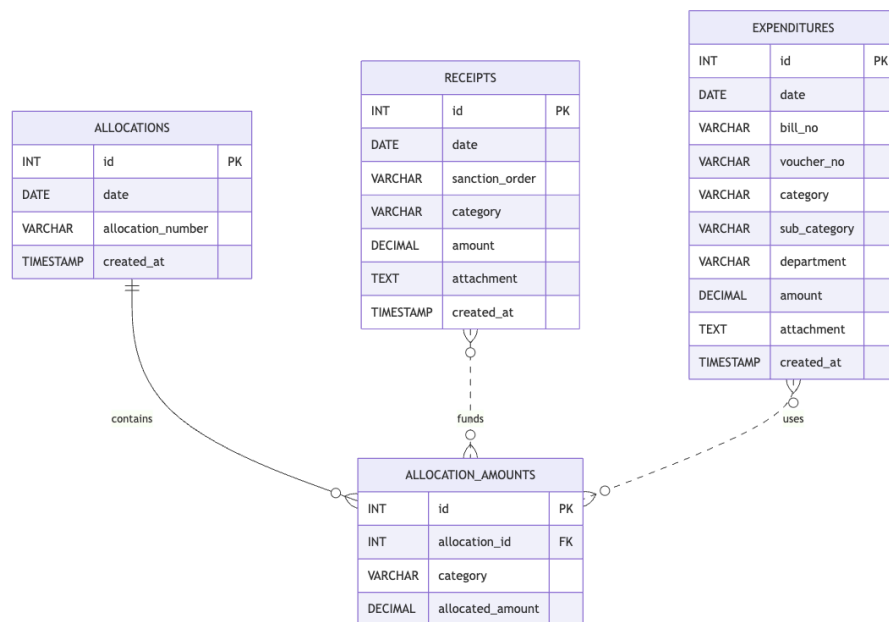


Fig. 9.4.1 ER Diagram Grants Dashboard

10. ECONOMIC ANALYSIS

Development of the Grants Dashboard is based on the use of completely free and open-source technologies, which helps eliminate the expenses related to licensing and subscriptions for commercial financial management tools.

In light of the fact that the platform should be deployed within existing organizational frameworks, there is no deployment cost at all. The Grants Dashboard can be run from existing organizational servers, as well as through regular web hosting services without having to purchase any commercial infrastructure.

The system offers higher flexibility when customizing it, centralized organizational control, and lower costs than traditional financial management systems used by enterprises.

Finally, the platform can help ease the organizational administrative burden associated with monitoring finances, generating reports and analyzing expenditures.

11. RESULTS AND DISCUSSION

11.1 Results

The Grants Dashboard effectively implemented the use of a centralized digital workflow for handling financial data regarding allocation, receipt, expenditure, and supporting documents.

Results included:

1. Analytics dashboard
2. Visualization of financial expenditures
3. Excel reporting
4. Access control
5. Responsive design

11.2 Challenges and Solutions

11.2.1 Cross-Origin Communication Problems

The initial communication between frontend and backend services faced some restrictions due to the cross-origin policy of browsers.

A CORS (Cross-Origin Resource Sharing) middleware was set up to solve the problem, which included credentials and proper validation of the origin for the secure exchange of information.

11.2.2 Responsive Dashboard Rendering

Initially, there were issues with rendering of the analytical charts on small screens and on mobile devices. For resolving the problem, a responsive design was added to the chart containers through Recharts.

12. CONCLUSION

The Grants Dashboard application clearly showed the feasibility of the idea behind an organization-oriented financial management solution involving allocation, receipt, expenditure, reporting, and analysis visualization.

It allowed improving the process of financial transparency, operations efficiency, workflow management, and minimizing manual data recording processes and other types of paperwork.

It showed the best practices from the modern world of full-stack software development, such as responsive UI design, API design using the REST approach, database access, analytical visualization features, security through authentication, and robust back-end solutions.

The adoption of open-source technologies guarantees sustainability, lower costs, and better scalability opportunities.

The current implementation of the Grants Dashboard allowed experiencing considerable improvement in workflow management, report creation, and analysis compared to administrative operations.

There is plenty of room for possible development in the Grants Dashboard system. For instance:

1. Granular permissions management for users
2. Predictive expenditure forecasting and budget management tools
3. Notification systems for emails and other means

REFERENCES

- [1] A. Robins, J. Rountree, and N. Rountree, "Learning and Teaching Programming: A Review and Discussion," *Computer Science Education*, vol. 13, no. 2, pp. 137–172, 2003.
- [2] M. Joy, N. Griffiths, and R. Boyatt, "The BOSS Online Submission and Assessment System," *Journal on Educational Resources in Computing*, vol. 5, no. 3, 2005.
- [3] LeetCode, "Online Coding Platform," 2025. Available: <https://leetcode.com>
- [4] HackerRank, "Technical Hiring and Skill Assessment Platform," 2025. Available: <https://www.hackerrank.com>
- [5] DOMjudge Team, "DOMjudge Documentation," 2025. Available: <https://www.domjudge.org/docs/manual/>
GitHub: <https://github.com/DOMjudge/domjudge>
- [6] M. K. Kwan and J. C. Cheung, "Self-hosted Educational Systems for Programming Assessment," *IEEE EDUCON*, 2021.
- [7] Vercel, "Next.js Documentation," 2025. Available: <https://nextjs.org/docs>
GitHub: <https://github.com/vercel/next.js>
- [8] D. Merkel, "Docker: Lightweight Linux Containers for Consistent Development and Deployment," *Linux Journal*, vol. 2014.
- [9] OWASP Foundation, "Authentication Cheat Sheet," 2025. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- [10] R. Sandhu et al., "Role-Based Access Control Models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [11] Docker Inc., "Docker Engine Documentation," 2025. Available: <https://docs.docker.com/engine/>

- [12] F. Ala-Mutka, "A Survey of Automated Assessment Approaches for Programming Assignments," *Computer Science Education*, 2005.
- [13] NIST, "Application Container Security Guide," SP 800-190, 2017. Available: <https://csrc.nist.gov/publications/detail/sp/800-190/final>
- [14] N. Provos and D. Mazieres, "Bcrypt Adaptive Hashing Algorithm," *USENIX*, 1999. Available: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf>
- [15] OWASP Foundation, "Session Management Cheat Sheet," 2025. Available: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- [16] React Team, "React Documentation," Meta, 2025. Available: <https://react.dev>
GitHub: <https://github.com/facebook/react>
- [17] T. Lin, "Server-Side Rendering with Next.js," *ACM Computing Surveys*, 2023.
- [18] Microsoft, "TypeScript Handbook," 2025. Available: <https://www.typescriptlang.org/docs/>
GitHub: <https://github.com/microsoft/TypeScript>
- [19] SQLite Consortium, "SQLite Documentation," 2025. Available: <https://www.sqlite.org/docs.html>
- [20] Docker Inc., "Container Security Best Practices," 2025. Available: <https://docs.docker.com/engine/security/>
- [21] UNESCO, "Digital Transformation in Higher Education," 2023. Available: <https://www.unesco.org/en/higher-education>
- [22] Intuit, "QuickBooks Product Documentation," 2025. Available: <https://quickbooks.intuit.com/learn-support/>
- [23] S. Few, "Information Dashboard Design," O'Reilly Media, 2013.
- [24] React Team, "React 19 Documentation," 2025. Available: <https://react.dev>
- [25] J. Garrett, "The Elements of User Experience," New Riders, 2011.

- [26] S. Wexler, J. Shaffer, and A. Cotgreave, *"The Big Book of Dashboards,"* Wiley, 2017.
- [27] Recharts Team, "Recharts Documentation," 2025. Available: <https://recharts.org/en-US>
GitHub: <https://github.com/recharts/recharts>
- [28] SheetJS Team, "SheetJS Documentation," 2025. Available: <https://docs.sheetjs.com>
GitHub: <https://github.com/SheetJS/sheetjs>
- [29] I. Sommerville, *"Software Engineering,"* 10th ed., Pearson, 2016.
- [30] R. Fielding, *"Architectural Styles and the Design of Network-based Software Architectures,"* Ph.D. dissertation, UC Irvine, 2000. Available: <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [31] Tailwind Labs, "Tailwind CSS Documentation," 2025. Available: <https://tailwindcss.com/docs>
GitHub: <https://github.com/tailwindlabs/tailwindcss>
- [32] PHP Group, "PHP Documentation," 2025. Available: <https://www.php.net/docs.php>
GitHub: <https://github.com/php/php-src>
- [33] Oracle Corporation, "MySQL Reference Manual," 2025. Available: <https://dev.mysql.com/doc/>
- [34] Recharts Team, "Recharts Visualization Library," 2025. Available: <https://recharts.org/en-US>
GitHub: <https://github.com/recharts/recharts>